

Benchmark

Aquí se encontrara exclusivamente el caso base que es probar 2 estrategias de algoritmos y establecer su comportamiento bajo el mismo estrés, pero de igual forma se añadirán otros casos en el espacio de “entregables”

```
import subprocess
import time
import csv

def run_benchmark():
    # Diferentes cargas de procesos para medir escalabilidad
    tamanos = [100, 500, 1000, 5000]
    resultados = []

    print("Iniciando Benchmark...")
    for n in tamanos:
        # Medimos el tiempo que tarda el ejecutable de C
        inicio = time.time()
        # Llamamos al ejecutable (asegúrate de que esté en bin/main)
        subprocess.run(["./bin/main.exe", str(n)], stdout=subprocess.DEVNULL)
        fin = time.time()

        tiempo_ms = (fin - inicio) * 1000
        resultados.append([n, tiempo_ms])
        print(f"Carga de {n} procesos completada en {tiempo_ms:.2f} ms")

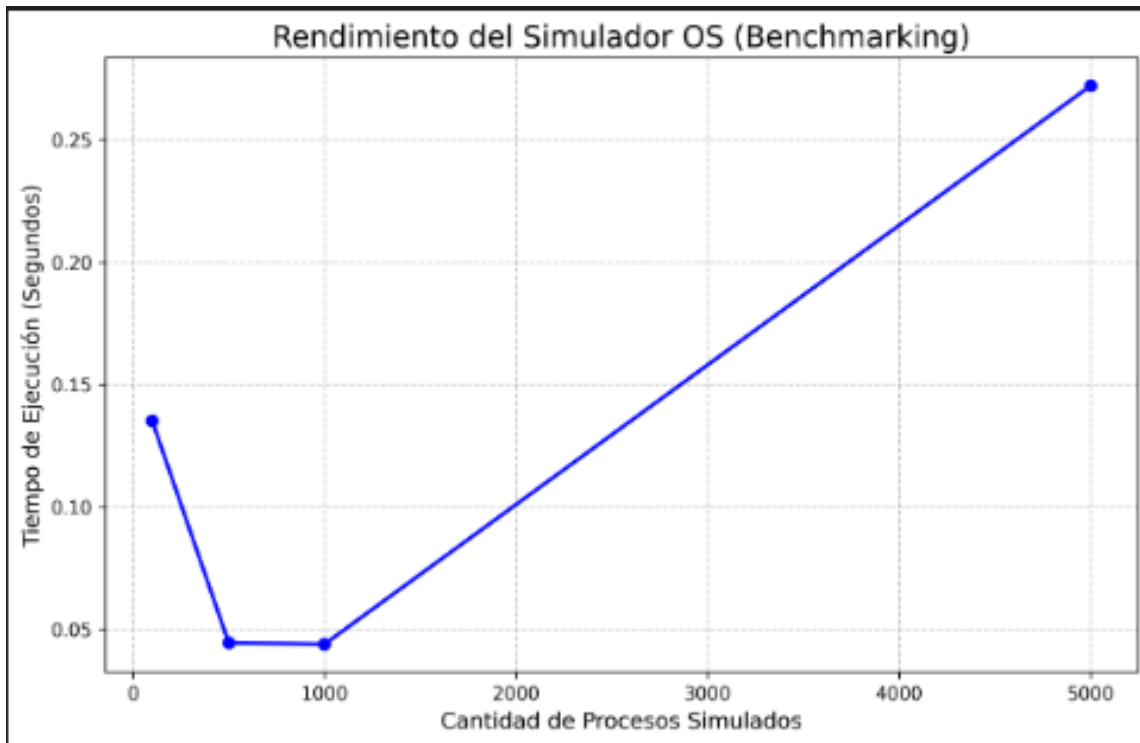
    # Guardamos los resultados
    with open("reports/csv/benchmark.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Procesos", "Tiempo_ms"])
        writer.writerows(resultados)
    print("Benchmark guardado en reports/csv/benchmark.csv")

if __name__ == "__main__":
    run_benchmark()
```

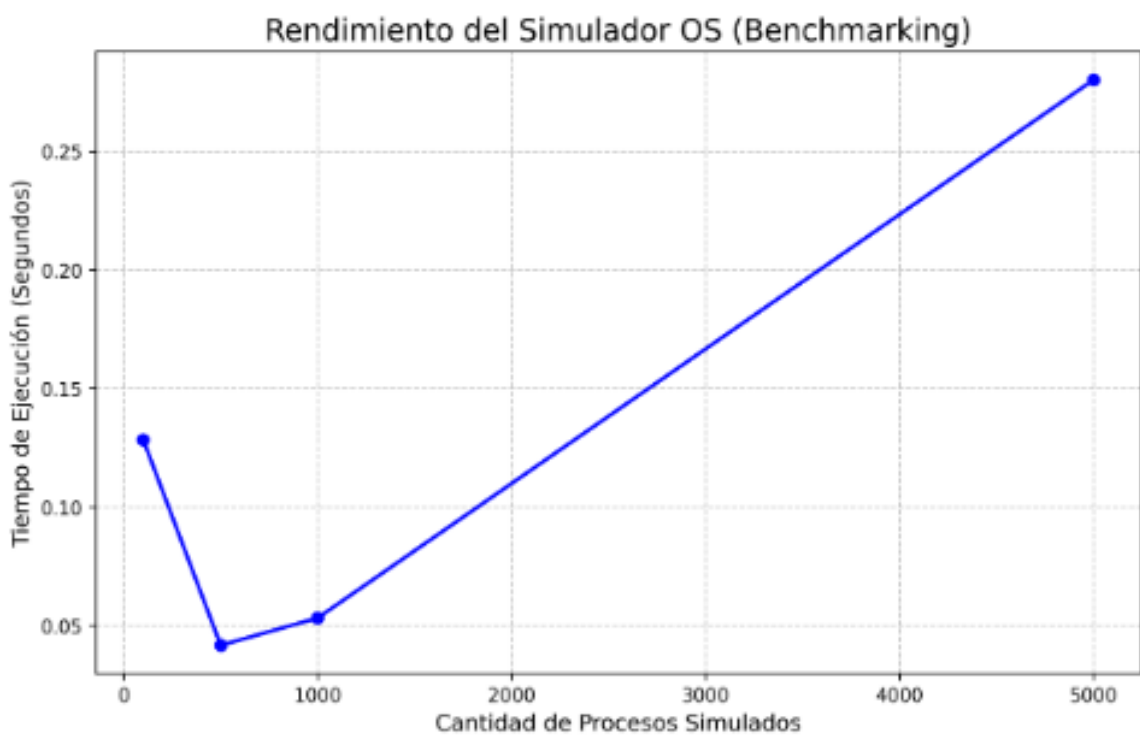
Graficas

Bajo las mismas condiciones de recursos, siendo una RAM de 1024 y Quantum de 2
Simulando 100, 500, 1000 y 5000 procesos

Con el first fit



Con el best fit



El first fit corresponde mejor a mayor cantidad de datos

Conclusion

Para esta prueba sencilla, se observan la diferencia entre los algoritmos en concreto, su velocidad a la hora de manejar datos tan grandes, aun que si son mas pequenos estos son básicamente iguales con la unica diferencia que el First Fit tiene una velocidad de “arranque” , como si fuera una computadora recién encendida, por ello es un poco menos veloz con datos pequenos

- **First Fit** es ligeramente más rápido (0.272s) porque simplemente recorre la memoria y en el momento que encuentra un hueco donde el proceso cabe, lo mete ahí y deja de buscar. Pierde menos tiempo procesando.
- **Best Fit** es un poco más lento (0.280s) porque, aunque encuentre un hueco rápido, tiene la obligación matemática de seguir buscando por **toda** la memoria para asegurarse de encontrar el hueco "más perfecto" (el que deje menos espacio desperdiciado). Esa búsqueda extra le cuesta tiempo de CPU.